Arduino-Einführung mit I/O

Worum geht es in diesem Blatt?

Sie arbeiten zum ersten Mal mit dem Arduino. Sie schreiben ein erstes Programm, schließen Sensoren und Aktoren an und lassen das Board eigenständig auf Eingaben reagieren. Zum Schluss kommt die drahtlose Kommunikation dazu. Damit haben Sie die Grundlage für alle weiteren Arduino-Blätter dieses Praktikums.

Was erwarten wir von der Dokumentation (Protokoll)?

- Beantworten Sie die gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Machen Sie zusätzlich Screenshots, Fotos usw., die Ihre Schaltungen in „Aktion“ zeigen
- Halten Sie Ihre **eigene Interpretation** fest: Was haben Sie im Experiment erkannt, was ist Ihnen aufgefallen?
- Welche **Probleme** sind Ihnen begegnet, und wie haben Sie sie **gelöst**?

Wie?

- **Bearbeiten Sie zur Abgabe den Test „Abgabe Praktikumsblatt 3“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie Ihre Erläuterungen und Ihren Quellcode in die jeweiligen Antwortfelder. Hängen Sie Screenshots und Bilder an, denn direkt ins Moodle-Quiz einfügen lassen sie sich in einigen Browsern nicht.**

Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Die Arduino-Plattform (`http://www.arduino.cc`) entstand 2005. Arduino ist eine Open-Source-Plattform für Elektronik(-projekte). Sie umfasst eine eigene Entwicklungsumgebung und verschiedene Input/Output-Boards mit programmierbaren Mikrocontrollern.

Arduino eignet sich besonders, um schnell Prototypen zu entwickeln. Die Hardware ist günstig, und die zugehörige Entwicklungsumgebung macht es sehr einfach, Programme zu schreiben und auf das Arduino-Board zu übertragen. In dieser Übung (und später in Kombination mit *vvvv*) verwenden Sie das Board „Arduino UNO R4 WiFi“ mit diversen Sensoren/Aktoren.

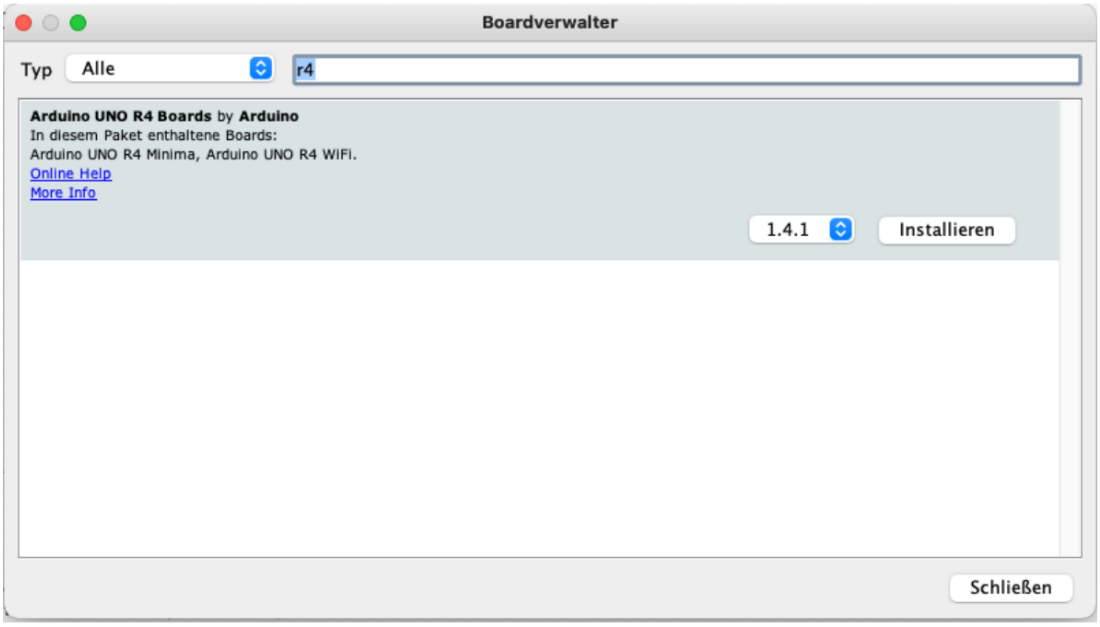
Nutzen Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Arduino Mikrocontroller“. Sie erklären Schritt für Schritt die Basisinformationen zu Arduino-Mikrocontrollern, zur Entwicklungsumgebung und zur Verwendung.

1) Mein erstes „Arduino-Programm“ (V4)

Im ersten Beispiel verwenden Sie das Dot-Matrix-Display, das auf dem Arduino Uno V4 Board integriert ist. Dazu installieren Sie zuerst die passende Bibliothek. Auf dem Praktikumsrechner sollte sie bereits verfügbar sein, auf Ihrem eigenen Rechner müssen Sie sie ggf. noch installieren.

Installieren und starten Sie die Arduino-Entwicklungsumgebung (https://www.arduino.cc/en/software/, auf den Praktikumsrechnern ist sie bereits vorinstalliert). Installieren Sie anschließend die passende Bibliothek für das R4 Board:

- Öffnen Sie den Boardverwalter: „Werkzeuge“ → „Board: Arduino XXX“ → „Boardverwalter...“
- Geben Sie im Suchfeld das Stichwort „R4“ ein und klicken Sie auf „Installieren“. Die Installation dauert meist ein paar Minuten. Legen Sie in der Zwischenzeit die Komponenten bereit, die Sie für dieses Aufgabenblatt brauchen (s. u.).



Sie brauchen folgende Komponenten

- Arduino UNO-R4 Board
- “*Breadboard*”
- Externe LED, rot (mit 2 Pins)
- Steckkabel (ca. 5 Stück)

Vorbereitung

- Starten Sie am PC die Arduino-Entwicklungsumgebung
- Verbinden Sie das Arduino-Board per USB-Kabel mit dem PC
- Konfigurieren Sie die Arduino IDE für den korrekten Chip (µC auswählen): Menü „Tools“ → „Microcontroller (MCU)“ → „Arduino Uno R4 Wifi“ auswählen

- Konfigurieren Sie die serielle Schnittstelle: Menü „Tools“ → „Serial Port“ → „COMX“ (den richtigen Port auswählen; falls Sie ihn nicht kennen: siehe Windows-Systemsteuerung)
- Legen Sie über „File“ → „New“ einen neuen sogenannten Sketch an

1.a) Ansteuerung der internen LED

Als Erstes bringen Sie die On-Board-LED des Arduino-Boards zum Blinken. So prüfen Sie auch, ob das Board funktioniert und die Installation korrekt ist. Die LED soll ein SOS-Signal senden. Das bedeutet:

- Die LED blinkt dreimal kurz („S“), dreimal lang („O“) und wieder dreimal kurz („S“).
- Nach dem SOS-Signal folgt eine Pause von drei Sekunden.
- Danach sendet die LED das SOS-Signal erneut (Endlosschleife).
- Bei einem kurzen Blinken leuchtet die LED 0,2 Sekunden, bei einem langen 0,6 Sekunden.
- Die Pausen innerhalb eines Buchstabens (LED aus) dauern ebenfalls 0,2 Sekunden, die Pause zwischen den Buchstaben 0,6 Sekunden.

Gegeben sind:

- Jedes Arduino-Programm enthält eine Setup-Methode (`void setup()`) für die Initialisierung (hier legen Sie auch fest, welche Pins zur Ein- oder Ausgabe dienen)
- Initialisiert wird mit dem Kommando `pinMode(Pinnummer, Modus)` – `Pinnummer`: siehe Beschriftung direkt am Arduino-Board; `Modus`: `INPUT` PIN wird gelesen, `OUTPUT` an den PIN wird etwas ausgegeben
- Dazu kommt eine `loop()`-Routine, die bis zum Programmende immer wieder durchlaufen wird (d. h., am Ende der loop()-Schleife springt das Programm wieder an ihren Anfang)
- Hier überträgt das Arduino-Board Eingaben an das Programm, und das Programm gibt Ausgaben an das Board
- Das Kommando `digitalWrite(Pinnummer, Wert)` gibt den entsprechenden Wert an den Pin `Pinnummer` aus. Zulässige Werte: `HIGH` (1, Ein), `LOW` (0, Aus)
- Der Befehl `delay(Dauer)` hält die Programmausführung für `Dauer` Millisekunden an
- Beim Arduino UNO-Board sprechen Sie die interne LED über Port 13 an
- **Wichtiger Hinweis**: In der Arduino IDE rufen Sie über das Kontextmenü eines markierten Befehls das Hilfesystem auf („*In der Referenz suchen*“) und bekommen detaillierte Informationen zur Verwendung dieses Befehls

Erstellen Sie mit diesen Informationen Ihr erstes Programm. Laden Sie dazu das Programm „`aufgabe_1a_vorlage.ino`“ von Moodle (die komplette zip-Datei **laden und entpacken**; den entpackten Ordner nicht unter „Desktop“ oder „Downloads“ ablegen) und ergänzen Sie die fehlenden Funktionen. Bevor Sie das fertige Programm auf das Arduino-Board spielen, wird es geprüft und kompiliert. Wählen Sie dazu in der Toolbar den zweiten Button („Hochladen“).

Frage 1: Funktioniert Ihr Programm? Was macht es?

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle)

Frage 2: Wie müssen Sie das Programm verändern, damit es ein SOS-Signal sendet? Geben Sie zusätzlich Ihren Programmcode ab.

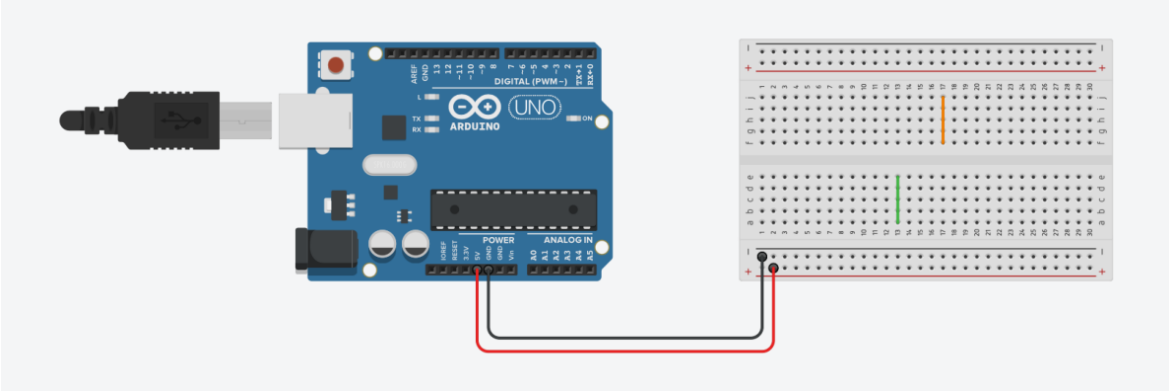
Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle)

1.b) Ansteuerung einer externen LED

Nehmen Sie jetzt die rote LED (mit 2 Pins). Sie soll an einem anderen Pin blinken, z. B. an Pin 9. Dazu passen Sie das Programm an. Stecken Sie zuerst das Arduino-Board vom USB-Stecker ab und bauen Sie die Schaltung auf dem „*Breadboard*“ auf.

Einschub: Was ist das „*Breadboard*“ (=Steckplatine, Steckbrett), und wie bauen Sie damit Schaltungen auf? Auf einem „*Breadboard*“ verbinden Sie elektronische Bauteile elektrisch, für Versuchsschaltungen und Experimente.

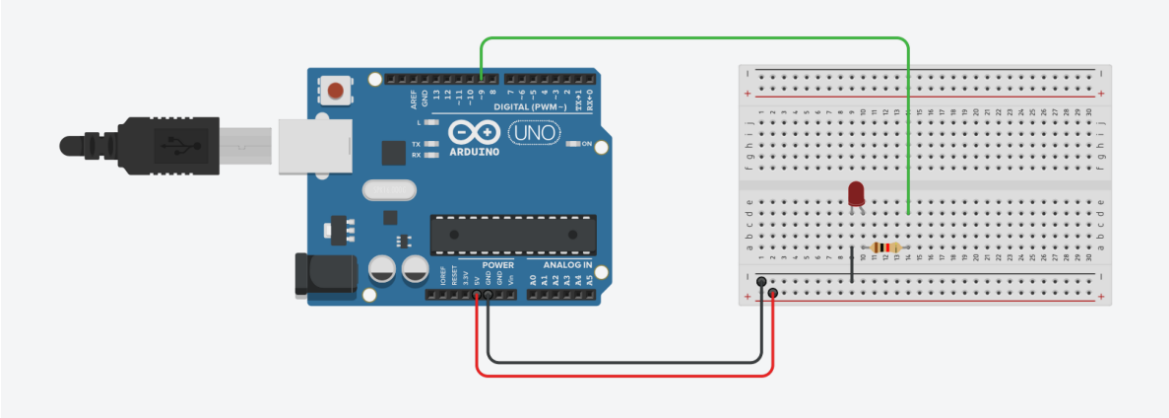
- Alle Pins in den beiden äußeren Reihen oben und unten sind horizontal miteinander verbunden. Dort legt man in der Regel Versorgungsspannung (+5 V) und Masse (GND) an und verteilt sie von dort einfach in die vertikalen Pinreihen
- Auch die vertikalen Reihen im Mittelteil sind jeweils intern miteinander verbunden, und zwar A-E und F-I; in der Abbildung unten sind z. B. die grün bzw. gelb markierten Pins miteinander verbunden



Diese Schaltung bauen Sie jetzt auf. Wie die Abbildung zeigt, brauchen Sie neben der LED ein zweites Bauteil, einen sogenannten „Vorwiderstand“. Bevor Sie die Schaltung aufbauen, rechnen Sie diesen Vorwiderstand aus.

Für die Berechnung finden Sie im Internet Widerstandsrechner, z. B. auf `https://www.leds-and-more.de/catalog/resistor.php/`

Hinweise zur Berechnung: Durchlassspannung der LED: 1,5 Volt, Betriebsstrom: 8 mA, das Arduino-Board liefert 5,0 Volt Betriebs- bzw. Versorgungsspannung



Frage 3: Wozu brauchen Sie diesen Vorwiderstand? Wie groß muss er sein (geben Sie die Berechnung an)?

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle)

Bauen Sie jetzt die Schaltung mit dem korrekten Vorwiderstand auf. (Lassen Sie sich von den Praktikumsbetreuern bestätigen, dass Ihre Berechnung stimmt, damit die LED nicht zerstört wird).

Frage 4: Was müssen Sie am Arduino-Programm ändern, damit die rote externe LED (an Pin 9) blinkt?

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

1.c) Dimmen von LEDs (mit Pulsweitenmodulation)

Vorweg: Anders als eine herkömmliche Glühbirne lässt sich eine LED auf „normale Weise“ nicht dimmen, nur ein- oder ausschalten. Schaltet man eine LED aber sehr schnell abwechselnd ein und aus („Pulsung“), nehmen Menschen eine geringere Helligkeit wahr. So lässt sich eine „Dimmung“ erreichen.

Gegeben sind:

- Jedes Arduino-Board hat Ports, die sich für Pulsweitenmodulation (PWM) nutzen lassen. Beim Arduino UNO sind das die Pins 3, 5, 6, 9, 10, 11
- PWM erzeugen Sie mit dem Kommando `analogWrite(Pinnummer, Wert)`. Achtung: Das funktioniert nur an einem PWM-Port (gekennzeichnet mit „~“)
- `Pinnummer`: siehe Beschriftung direkt am Arduino-Board (jene mit „~“)
- `Wert`: Gibt den sogenannten „duty cycle“ an und liegt zwischen 0 (immer aus) und 255 (2^8-1, immer ein)
- Am Ausgang entsteht ein kontinuierliches Rechtecksignal mit dem entsprechenden Puls-Pausen-Verhältnis (Frequenz: 490/980 Hz)

Lassen Sie die LED an Pin 9 jetzt pulsweitenmoduliert mit verschiedenen Helligkeitswerten leuchten (z. B. 50, 100, 200). Verändern Sie dazu das vorherige Programm. Kompilieren Sie das Programm nach jeder Änderung und übertragen Sie es auf das Arduino-Board.

Frage 5: Funktioniert Ihr Programm? Was stellen Sie fest? Wie funktioniert Pulsweitenmodulation (PWM)? Skizzieren Sie es. Geben Sie auch Ihren Programmcode ab.

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

Etwas dynamischer wird es, wenn die LED „pulst“: Die Helligkeit steigt von 0 (=aus) langsam bis zur vollen Helligkeit und nimmt dann wieder langsam ab. Das Arduino-Programm „`aufgabe_1c_vorlage.ino`“ laden Sie wieder aus Moodle.

Ergebnis: eindeutig ein „schöneres“ Blinken, finden Sie nicht? Experimentieren Sie jetzt mit den Werten für `fadeAmount` und `delay` (kompilieren Sie das Programm nach jeder Änderung und spielen Sie es auf das Arduino-Board).

Frage 6: Wie funktioniert das Programm? Was beobachten Sie?

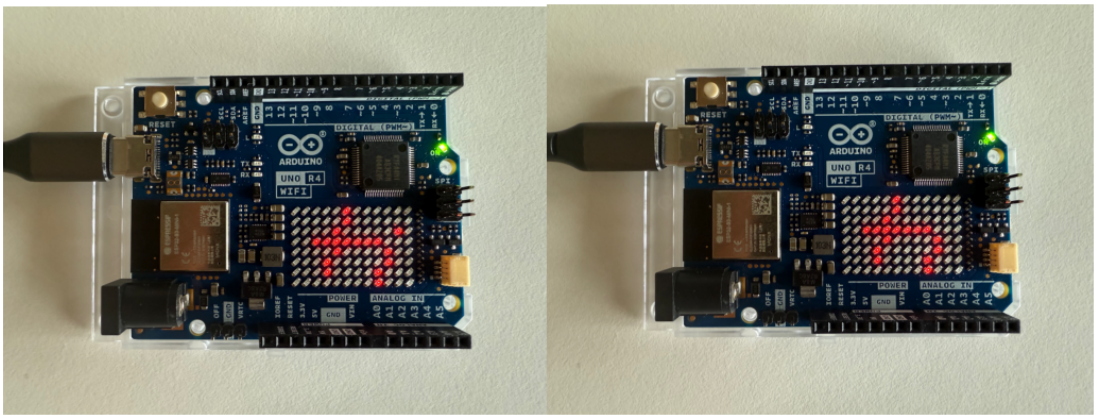
Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

1.d) Ansteuerung des internen Displays

Installieren Sie anschließend die Bibliothek für das LED Matrix Display:

- Öffnen Sie die Bibliotheksverwaltung: „Werkzeuge“ → „Bibliotheken verwalten“
- Geben Sie im Suchfeld das Stichwort „HT16K33“ ein und klicken Sie auf „Installieren“.

Jetzt soll das interne Display das THI-Logo zeigen. Das Breadboard brauchen Sie für diese Aufgabe nicht, Sie können es also vom Arduino abstecken. Lassen Sie die Bauteile aber auf dem Breadboard stecken: Sie brauchen das Board in Aufgabe 2 noch. Am Ende dieser Aufgabe sieht das Programm so aus:



Das Display zeigt das THI-Logo, und der Punkt auf dem „i“ soll regelmäßig blinken. Öffnen Sie zuerst das Arduino-Programm „`aufgabe_1d_vorlage.ino`“. Es lässt sich bereits ausführen.

Frage 7: Wie funktioniert das Programm? Was beobachten Sie?

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

Versuchen Sie zu verstehen, wie die einzelnen Frames aufgebaut sind. Im Arduino-Programm ist ein Frame so codiert:

```
{
  0x80000000,
  0x00000000,
  0x00000000
}
```

Tipp: Verschieben Sie die Zahl „8“ Stück für Stück nach rechts und auch in die Zeilen 2 und 3. Verwenden Sie statt der Zahl 8 testweise auch die Zahlen 1, 2 und 4. Was beobachten Sie? Was passiert, wenn Sie andere (hexadezimale) Zahlen zwischen 0 und f verwenden?

Frage 8: Definieren Sie in Ihrem Programm zwei Frames, sodass die ersten vier LEDs und die letzten vier LEDs abwechselnd blinken.

Dokumentieren Sie den Programmcode in ihrem Protokoll (auf Moodle)

Jetzt erstellen Sie die zwei Varianten des THI-Logos. Versuchen Sie zuerst, die einzelnen LEDs über das Array anzusteuern. Gelingt Ihnen das mit Ihrem Programmcode nicht, hilft der LED Matrix Editor (`https://ledmatrix-editor.arduino.cc`). Dort ist ein Frame standardmäßig in vier Zeilen codiert. Die vierte Zeile brauchen Sie hier nicht und können sie entfernen. Sie stört die Darstellung aber auch nicht: Maßgeblich sind nur die Zeilen 1 bis 3 Ihres Frames.

Frage 9: Erstellen Sie ein Programm, das ein animiertes THI-Logo darstellt.

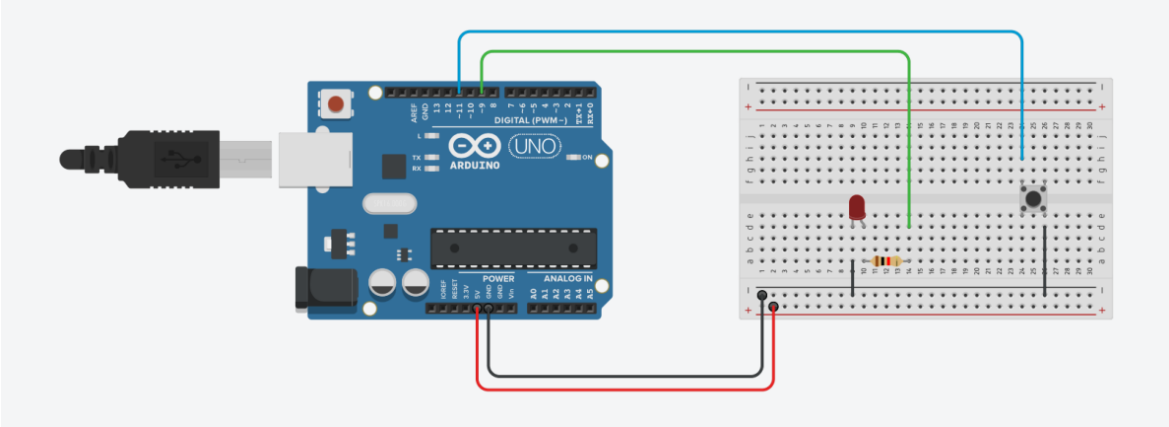
Dokumentieren Sie den Programmcode in ihrem Protokoll (auf Moodle)

2) Arduino als intelligenter Controller

Hier betreiben Sie das Arduino Board als Standalone-Controller für Ein- und Ausgabe. Die komplette Logik läuft auf dem Board; mit externer Spannungsversorgung bräuchten Sie weder USB-Kabel noch Computer. Computer und USB-Kabel dienen also nur noch der Spannungsversorgung.

2.a) Button steuert LED

In der folgenden Schaltung und dem zugehörigen Programm steuert ein „Push-Button“ direkt die LED. Stecken Sie das Arduino-Board wieder vom USB-Anschluss ab und erweitern Sie die Schaltung auf dem „*Breadboard*“ wie unten dargestellt.



Noch ein Hinweis zu den Drahtfarben in der Schaltung: „Schwarz“ kennzeichnet in der Regel „Masse“ (0 Volt, GND), „rot“ die Versorgungsspannung (hier: +5 Volt); alle anderen Farben übertragen Signale von Sensoren usw.

Der restliche Aufbau:

- Greifen mehrere Komponenten auf Masse oder Versorgungsspannung zu, legt man diese beiden Signale auf die äußeren Reihen des „*Breadboards*“: Alle Pins der 4 äußeren Reihen sind (pro Reihe) miteinander verbunden.
- Die LED bleibt über den Vorwiderstand (Größe wie zuvor) an Pin 9 angeschlossen.
- Schließen Sie den „Push-Button“ wie auf dem Schaltplan gezeigt an. Die beiden linken Pins sind fest miteinander verdrahtet; ob Sie das blaue Kabel links unten oder links oben anschließen, ist also egal. Dasselbe gilt für die beiden rechten Pins.
- Ist der Button nicht gedrückt, besteht keine Verbindung zwischen seiner linken und rechten Seite (also zwischen dem blauen und dem schwarzen Kabel). Drücken Sie den Button, verbindet er diese beiden Anschlüsse (PIN 11 wird also „auf Masse gezogen“).

Öffnen Sie jetzt das Programm `aufgabe_2a_vorlage.ino`. Am Anfang werden die Variablen buttonPin und ledPin definiert. Wenn Sie LED oder Button an anderen Pins anschließen, passen Sie diese Variablen an.

In der `setup()`-Methode legen Sie die Zuordnungen fest (Button = Eingabe , LED = Ausgabe), d. h., ein Pin empfängt Signale, der andere sendet Signale. Ist der Button offen, ist er weder mit GND noch mit VCC verbunden. Er „floatet" also und würde zufällige Werte liefern, wenn Sie ihn als „normalen" `INPUT` definieren. Mit pinMode `INPUT_PULLUP` dagegen nutzt der Arduino einen internen, sogenannten PullUp-Widerstand. Das bedeutet: Ist der Button offen, liefert der Arduino an diesem Pin konstant einen `HIGH` -Pegel.

In der loop()-Routine gehen Sie so vor: Lesen Sie den Status des Buttons ein, speichern Sie den Wert in der Variablen „`buttonState`" und steuern Sie damit, abhängig vom Wert (LOW bzw. HIGH), die LED an. Wichtig: Das Verhalten ist genau invertiert. Wird der Button gedrückt, ist er LOW, und die LED soll leuchten (HIGH). HIGH und LOW sind Konstanten, Sie können sie also (ohne Anführungszeichen) in Ihrem Code verwenden.

Der „Push-Button" ist an Pin 11 angeschlossen, einem digitalen Pin. Den Status lesen Sie mit der Methode `erg = digitalRead(pin)` ein (und setzen dort die korrekten Variablennamen für `erg` und `pin` ein). Je nach Wert schalten Sie die LED ein oder aus, wieder mit der Funktion `digitalWrite(pin, wert)`. Sie können mit einer Fallunterscheidung (if / else) arbeiten oder die Negation (`!wert`) verwenden.

Stellen Sie Schaltung und Programm fertig, kompilieren Sie Ihre Lösung und laden Sie sie auf das Arduino Board.

Frage 10: Funktioniert die Interaktion wie gewünscht? Wie sieht Ihr finaler Code aus? Kopieren Sie Ihre Lösung wieder in das Protokoll bzw. den Upload. Gerne können Sie zusätzlich ein Foto Ihrer Schaltung (Arduino+Breadboard+Verkabelung) mit abgeben.

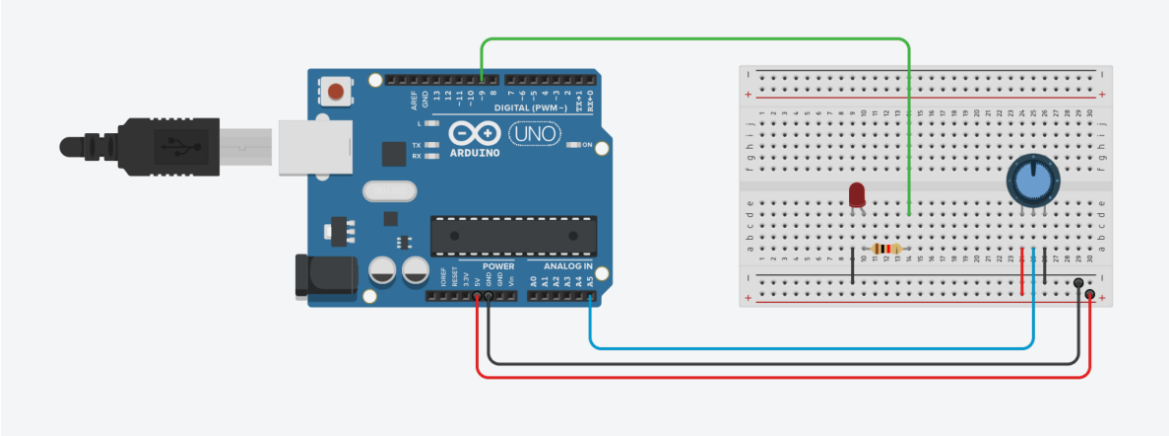
Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

2.b) Potentiometer steuert LED-Helligkeit

Als Nächstes verändern Sie mit einem Drehregler („Potentiometer") direkt und linear die Helligkeit einer LED. Vorgehensweise:

Stecken Sie das Arduino-Board wieder vom USB-Anschluss ab und ändern Sie die vorherige Schaltung auf dem „*Breadboard*" wie unten dargestellt. Die Potentiometer gibt es in zwei Ausführungen: in der abgebildeten schwarzen Variante und als etwas kompakteres blaues Bauteil. Die blauen funktionieren etwas robuster.

So funktioniert der Drehregler: Legen Sie den linken Pin an die Versorgungsspannung (VCC, +5 V), den rechten an Masse (GND, 0 V). Am mittleren Pin greifen Sie die Spannung des Spannungsteilers ab, je nach Stellung des Potentiometers (kontinuierlicher Wert von 0 bis +5 Volt). Diesen analogen Wert verwenden Sie über einen Analogeingang am Arduino-Board im Programm weiter (A0 bis A5). Die LED bleibt über den Vorwiderstand (Größe wie zuvor) an Pin 9 angeschlossen



Öffnen Sie jetzt das Programm „`aufgabe_2b_vorlage.ino`“, das die Logik abbildet. Auch hier gibt es wieder einige globale Variablen (noch vor der `setup()`-Methode): eine Variable für den Eingabepin des Drehreglers, der am Anschluss A5 hängen soll, und eine Variable für den Ausgabepin der LED, wie zuvor Pin 9.

In der `setup()`-Methode legen Sie die Zuordnungen fest (Drehregler = Eingabe , LED = Ausgabe), d. h., Pin A5 empfängt Signale, Pin 9 sendet Signale. In der `loop()`-Routine lesen Sie zuerst den Wert des Potentiometers ein und speichern ihn in der Variablen, die Sie dafür definiert haben. Bevor Sie damit die LED ansteuern, müssen Sie ihn umrechnen:

Den Wert des Drehreglers lesen Sie mit der Funktion `erg = analogRead(pin)` ein (erg, pin sind Platzhalter für „unsere“ Variablen). Das Board kann nicht direkt mit analogen Spannungen arbeiten. Es „digitalisiert“ sie mit einem sogenannten Analog-Digital-Konverter (ADC). Der Arduino UNO R4 hat einen 14-Bit-ADC, d. h., er bildet den Eingangsbereich von 0 bis 5 Volt auf Integer-Werte von 0 bis 16.383 ab (2^{14} = 16.384 Stufen). Die Variable nimmt also Werte von 0 bis 16.383 an.

Aus dem Beispiel zur Pulsweitenmodulation wissen Sie: Mit der Funktion `analogWrite(Pinnummer, Wert)` „dimmen“ Sie eine LED am Ausgang. Der Wertebereich für Wert liegt weiterhin zwischen 0 und 255 (= 8 Bit). Damit der Eingabewert des Potentiometers die LED direkt dimmen kann, rechnen Sie den Wertebereich 0 bis 16.383 (des ADC) auf 0 bis 255 (für PWM) um. Das erledigt die Funktion `map()`, z. B. `erg = map(eingabe, 0, 16383, 0, 255)`.

Stellen Sie die Schaltung fertig, kompilieren Sie Ihre Lösung und laden Sie sie auf das Arduino Board. Der Drehregler hält unter Umständen nicht besonders gut am „*Breadboard*“. Abhilfe schaffen 30cm „Male-Female Extenderkabel“ aus dem „Prototyping Interfaces Starterkit“.

Frage 11: Funktioniert die Interaktion wie gewünscht? Gerne können Sie zusätzlich ein Foto Ihrer Schaltung (Arduino+Breadboard+Verkabelung) mit Ihrer Lösung abgeben.

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

Eine Möglichkeit für „Debugging“: Das Arduino-Board hat kein großes Display, auf dem Sie alle Werte ausgeben könnten. Zum Debuggen brauchen Sie also einen anderen Weg. Meist interessiert Sie der Inhalt bestimmter Variablen. Den geben Sie auf die serielle Schnittstelle aus und überwachen ihn im sogenannten „Serial Monitor“. Das ist eine einfache Möglichkeit für „Debugging“, und so geht’s:

Fügen Sie in der `setup()`-Methode folgende Zeile ein, um eine serielle Verbindung zu öffnen: `Serial.begin(115200);`. In der `loop()`-Methode geben Sie mit `println` (oder `print`) Inhalte auf der seriellen Schnittstelle aus: `Serial.println(„Text“);`

Über das Menü „Tools“ → „Serial Monitor“ überwachen Sie jetzt den Inhalt der Variablen, während das Programm auf dem Arduino läuft (und der Arduino per USB-Kabel mit dem PC verbunden ist). Voraussetzung: Sie haben dort dieselbe Baud-Rate (`115200`) eingestellt.

Frage 12: Erweitern Sie Ihr Programm um diese Monitoring-Funktion und fügen Sie den geänderten Code und einen Screenshot des Serial Monitors hier ein.

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll

3) Drahtlose Kommunikation mit dem Arduino

Hier verbinden Sie einen Arduino UNO R4 WiFi ohne zusätzliches WLAN mit einem Computer, und zwar über eine selbst aufgebaute WLAN-Verbindung. Der Arduino arbeitet als Access Point und bietet einem verbundenen Gerät (z. B. einem Java-Programm auf einem PC) eine direkte TCP/IP-Verbindung an.

Der Vorteil: Sie brauchen kein zusätzliches Netzwerk (z. B. keinen Router) und können sofort direkt kommunizieren, etwa um Daten zu steuern, abzufragen oder zu visualisieren.

3.a) Arduino als Access Point und TCP-Server

Im ersten Schritt programmieren Sie den Arduino so, dass er ein eigenes WLAN-Netz bereitstellt. Benennen Sie dieses Netz z. B. nach Ihren beiden Nachnamen (z. B. „`RienerNestler`“) und sichern Sie es mit einem Passwort (mit mindestens **acht Zeichen**). Außerdem lauscht der Arduino auf einem bestimmten Port (hier: Port `8888`) und nimmt dort eingehende Verbindungen von Clients an.

Öffnen Sie jetzt das Programm `aufgabe_3a_arduino_vorlage.ino`. Legen Sie am Anfang die SSID (Netzwerkname) und das Passwort für das WLAN fest.

Sobald der Access Point aktiv ist, bekommt er eine feste IP-Adresse (standardmäßig 192.168.4.1). Über diese IP kommuniziert ein verbundenes Gerät mit dem Arduino. Der Sketch startet einen Server auf Port 8888. In der loop()-Methode wartet der Arduino auf eingehende TCP-Verbindungen. Verbindet sich ein Client und schickt Daten, empfängt der Arduino sie per readStringUntil('\n') und gibt sie über den Serial Monitor aus.

Frage 13: Verbinden Sie Smartphone und/oder PC mit dem WLAN, das Sie gerade erzeugt haben, und erstellen Sie einen Screenshot der erfolgreich hergestellten Verbindung.

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle).

3.b) Java-Client zur Kommunikation mit dem Arduino

Als Gegenstück zum Arduino brauchen Sie eine Client-Anwendung, die über WLAN mit dem Access Point des Arduino verbunden ist und Daten senden kann. Das übernimmt ein einfaches Java-Programm mit grafischer Benutzeroberfläche.

Öffnen Sie das Projekt `aufgabe_3b_java_vorlage`. Dieses Programm verbindet sich über einen TCP-Socket mit der IP-Adresse `192.168.4.1` und Port `8888` (beide Werte sind fest im Programm hinterlegt). Nach dem Start geben Sie in ein Eingabefeld ein Zeichen oder einen Text ein und übertragen ihn mit einem Klick auf „Senden“ an den Arduino.

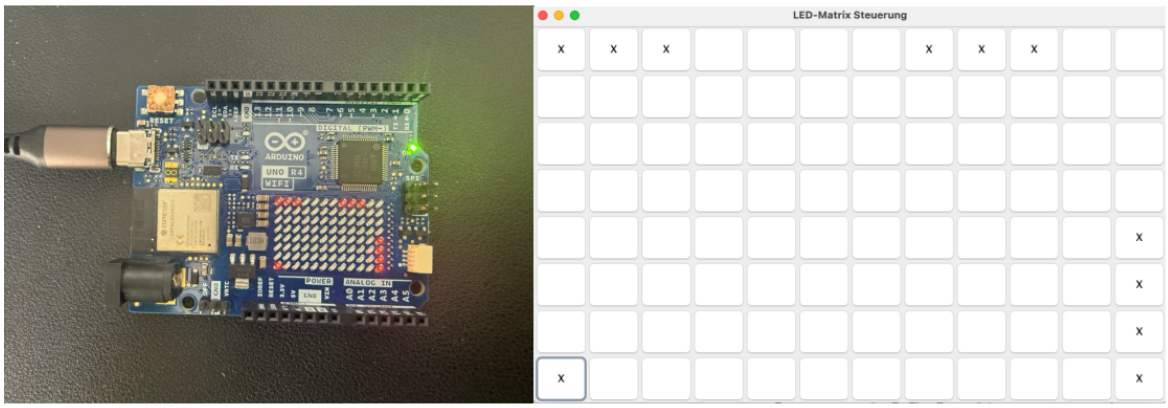
Das Programm baut die Verbindung bei jedem Klick neu auf und schließt sie nach dem Senden sofort wieder. Der Arduino liest den empfangenen Text zeilenweise ein (`readStringUntil('\n')`) und zeigt ihn im Serial Monitor an.

Frage 14: Funktioniert die Kommunikation zwischen dem Java-Client und dem Arduino wie gewünscht? Kopieren Sie Ihren finalen Arduino-Code in das Protokoll. Gerne können Sie zusätzlich ein Foto Ihrer Testumgebung und einen Screenshot des Serial Monitors (mit den empfangenen Zeichen) einfügen.

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (auf Moodle).

3.c) Java-GUI zur Steuerung der LED-Matrix

In dieser erweiterten Aufgabe kombinieren Sie mehrere Techniken: Sie steuern die integrierte 12×8-LED-Matrix des Arduino UNO R4 WiFi vollständig drahtlos über WLAN, mit einer selbst erstellten grafischen Java-Anwendung. Ziel: den aktuellen LED-Zustand aus der Benutzeroberfläche auslesen, als kompakten Datenstring kodieren und an den Arduino senden. Kodiert wird in drei `32-Bit-Hexwerten`, die den Zustand der 96 LEDs (8 Zeilen × 12 Spalten) vollständig abbilden.



Für das Arduino-Setup verwenden Sie das bereitgestellte Programm `aufgabe_3c_arduino_vorlage.ino`. Der Arduino spannt ein eigenes WLAN auf und wartet über Port `8888` auf eingehende Daten. Sobald er ein gültiges Datenpaket empfängt (drei durch Leerzeichen getrennte 8-stellige Hex-Zahlen), übergibt er es direkt an die LED-Matrix und zeigt es an. Dieses Arduino-Programm müssen Sie nicht mehr anpassen.

Auf Java-Seite nutzen Sie das Projekt `aufgabe_3c_java_vorlage`. Die Benutzeroberfläche besteht aus einem 12×8-Raster von Schaltflächen; jede Schaltfläche entspricht genau einer LED auf der Matrix. Ein Klick auf eine Schaltfläche schaltet die entsprechende LED ein oder aus. Die Oberfläche zeigt den aktuellen Zustand direkt an: Eine aktive LED erscheint als „X“, eine inaktive bleibt leer.

Nach jeder Änderung baut das Programm den vollständigen Matrix-Zustand intern als binäre Zeichenkette auf (96 Bits = 12 Spalten × 8 Zeilen). Diese Bitfolge wandelt es in 24 Hex-Zeichen um (4 Bits = 1 Hex-Zeichen). Zuletzt zerlegt es den Hex-String in drei Teile zu je 8 Zeichen (3 × 32 Bit) und überträgt sie als Datenpaket an den Arduino.

Ihre Aufgabe: Implementieren Sie in der Methode `sendMatrixState()` die fehlenden Teile des Programms. Die Grundstruktur steht bereits, es fehlen aber die zentralen Verarbeitungsschritte. Die Stellen sind im Code als `TODOs` markiert. Überführen Sie den aktuellen LED-Zustand korrekt in eine 96-Bit-Folge, wandeln Sie sie in Hex um und senden Sie die drei resultierenden Datenblöcke über die TCP-Verbindung an den Arduino. Solange Ihre Implementierung unvollständig ist, läuft das Programm trotzdem, mit Platzhalterwerten: Es zeigt dann immer einen mittleren Block mit aktiven LEDs und zwei inaktive Ränder.

Bearbeiten Sie alle `TODOs` Schritt für Schritt und testen Sie Ihr Programm anschließend mit einer aktiven WLAN-Verbindung zum Arduino.

Frage 15: Kodierung verstehen. Beschreiben Sie in eigenen Worten, wie der Zustand der 12×8-LED-Matrix intern gespeichert und für die Übertragung auf drei 32-Bit-Zahlen aufgeteilt wird. Warum ist diese Art der Kodierung sinnvoll?

Frage 16: Anwendung testen und reflektieren. Testen Sie Ihre Lösung, indem Sie ein bestimmtes Muster (z. B. ein „H“, ein Rahmen oder eine diagonale Linie) auf der LED-Matrix darstellen. Fügen Sie einen Screenshot der Benutzeroberfläche und einen Screenshot des Serial Monitors bei. Was würden Sie an der Benutzeroberfläche oder Datenübertragung verbessern?

BITTE BEACHTEN: Am Ende der Praktikumsstunde…

- …bauen Sie die Schaltungen wieder ab
- …verpacken Sie sämtliche Bauteile wieder sauber und vollständig in der Schachtel: Widerstände zurück in die richtigen Fächer, Drehregler/LEDs in ihre Fächer usw.